IT5001 / PRACTICAL PYTHON

DATA TYPES · OPERATIONS · CONVERSIONS · RETURN VALUES

01 Numbers · bool · None

`n=0; x=0.0; ok=False; v=None`

`int(x)` truncates toward 0; `float(x)` → float; `str(x)` → text; `bool(x)` → truth value.

```
int('12')        # 12
int(-3.9)        # -3
int('2.5')       # ValueError
```

`+ - * / // % **` • `/` gives float for integers; `//` floors; `%` has the divisor's sign.

```
-7 // 3          # -3
-7 % 3           # 2
angle % 360      # 0..359 for int angle
n % 10           # last digit (n >= 0)
n // 10          # drop last digit
```

`abs(x)` • `round(x)` → int for float x; `round(x, ndigits)` → rounded value. Ties round to even; floats may be approximate.

`min(a,b,...)` / `max(a,b,...)` → an argument. `min(it, key=f, default=d)` / `max(...)`: key=None by default; `default` only with iterable form.

`sum(it, start=0)` → total; `sum([])` → 0. Empty `min/max` without `default` → `ValueError`.

False: `False`, `None`, numeric zero, empty containers/strings. `True == 1`; `False == 0`.

`and` returns first falsy operand, otherwise last; `or` returns first truthy operand, otherwise last. `not x` → bool. Right side may be skipped.

`== != < <= > >=` • `0 <= i < n` • `x is None`. `==` compares values; `is` compares identity.

Precedence: `**` → unary `+ -` → `* / // %` → `+ -` → comparisons → `not` → `and` → `or`.
`-2**2 == -4; 2**-2 == 0.25; 2**3**2 == 512`.

02 Indexed sequences · str / list / tuple

`a[i]` → item; indices `0..n-1` or `-n..-1`. Invalid index → `IndexError`; slices clip to bounds.

```
a[start:stop:step]  # stop excluded
a[:k]               # first k items
a[k:]               # from k to end
a[1:-1]             # remove both ends
a[::2]              # every second item
a[::-1]             # reversed sequence
```

Defaults for positive step: start 0, stop n. For negative step: last item to before first. Step `0` → `ValueError`; wrong direction → empty.

```
'abcde'[4:0:-1]    # 'edcb'
'abcde'[4::-1]     # 'edcba'
'abcde'[:-1:-1]    # ''
```

`len(a)` → int; `x in a` / `x not in a` → bool. Strings: substring test. Lists/tuples: whole item test; membership is shallow.

`a + b`: concatenate same sequence type. `a * n`: repeat; `n <= 0` gives empty sequence.

Same-type strings/lists/tuples compare lexicographically: first unequal item decides; equal prefix → shorter comes first.

```
'Z' < 'a'          # True
[1, 9] < [2]       # True
(1, 2) < (1, 2, 0) # True
[1] == (1,)        # False
```

Ordering incompatible types (e.g. `1 < "2"`) → `TypeError`. Nested elements must also be comparable.

03 Dictionaries · dict

Mutable; unique keys; insertion order.
`d = {}` / `dict()` • `d = {"a":2}`
`dict([("a",2),("b",3)])` → pairs to dict.
Keys must be hashable; values can be any type.

```
d[k] = v           # insert / overwrite
d[k]               # value; KeyError if absent
d.get(k)           # value or None
d.get(k, default)  # value or default
k in d             # checks KEYS
d.pop(k)           # remove/return; KeyError
d.pop(k, default)  # default if absent
del d[k]           # remove; KeyError if absent
d.clear()          # empty; returns None
d.copy()           # shallow dict copy
```

`len(d)` → number of pairs; `d.keys()` / `d.values()` / `d.items()` → live views (not lists).

```
for k in d: ...
for v in d.values(): ...
for k,v in d.items(): ...
list(d)            # list of keys
list(d.items())    # list of (key,value)
```

`d1 == d2` compares key/value pairs, ignoring order; `< >` unsupported. Changing size during iteration → `RuntimeError`. Dictionary sorting: p2.

Frequency count · anagrams compare counts

```
counts = {}
for x in data:
    counts[x] = counts.get(x,0) + 1
```

04 Strings · str

Immutable • `s = ""` / `str()` • `s = "abc"`.
No item assignment, append, remove or pop.
String operations return new strings: `s = s.lower()`.

Case · test · compare

```
s.lower()              # lowercase str
s.upper()              # uppercase str
'a' <= c <= 'z'        # ASCII lower letter
'A' <= c <= 'Z'        # ASCII upper letter
'0' <= c <= '9'        # ASCII digit
```

Range tests assume `len(c) == 1`. Whole string: loop over its characters. Comparison is case-sensitive.

Split → list[str]

```
s.split(sep=None, maxsplit=-1)
' a  b\n'.split()    # ['a','b']
'a,,b,'.split(',')   # ['a','','b','']
'a,b,c'.split(',',1) # ['a','b,c']
''.split()           # []
''.split(',')        # ['']
```

No separator: runs of whitespace collapse; no empty edge fields. Explicit separator: empty fields remain. `maxsplit=-1`: no limit; empty separator → `ValueError`.

Trim · replace → str

```
s.strip()              # whitespace, both ends
s.strip(chars)         # given chars, both ends
s.lstrip(chars)        # left only
s.rstrip(chars)        # right only
'xyhelloyx'.strip('xy') # 'hello'
```

`chars` is a collection of characters, **not** a whole prefix/suffix. Omit it / use None for whitespace. Interior characters remain.

```
s.replace(old, new)      # all matches
s.replace(old, new, n)   # at most n
'banana'.replace('a','',2) # 'bnna'
'aaa'.replace('aa','X')   # 'Xa'
```

Non-overlapping matches, left to right. Count is positional: negative → all; 0 → unchanged; missing match → unchanged.

Add · remove · build

```
s += 'x'             # rebind to s + 'x'
s = s[:i] + 'X' + s[i:]  # insert at i
s = s[:i] + s[i+1:]      # delete at i
s = s[:i] + 'X' + s[i+1:] # replace i
```

Slice recipes above assume `0 <= i < len(s)`; insertion also allows `i == len(s)`.

```
s.replace('x','')         # remove all x
''.join([c for c in s if c != 'x'])
'-'.join(['a','b'])       # 'a-b'
''.join(map(str,[1,2]))   # '12'
```

`sep.join(it)` needs strings; empty iterable → `""`. `list("ab")` → `["a","b"]`; `tuple("ab")` → `("a","b")`; `str([1,2])` → `"[1, 2]"`, not `"12"`.

Find · count · prefixes

`s.find(sub)` / `s.find(sub,start,end)` → first index or -1; `s.index(...)` → index or `ValueError`. `s.count(sub)` / `s.count(sub,start,end)` → non-overlapping count. Optional positional bounds: start 0, end `len(s)`; end excluded.

```
s[:len(p)] == p       # starts with p
s[-len(p):] == p      # ends with p; p != ""
```

05 ASCII · character conversions

`ord(c)` → integer code (one character).
`chr(n)` → character. Python strings are Unicode; ASCII codes below are the common subset.

```
ord('0') == 48; ord('A') == 65
ord('a') == 97; chr(97) == 'a'
ord(c)-ord('a')       # a..z -> 0..25
chr(ord('a') + i)     # 0..25 -> a..z
```

Decimal codes: `0..31` controls • 9 tab `\t` • 10 newline `\n` • 13 carriage return `\r` • 32 space • 127 DEL.

```
33..47   !"#$%&'()*+,-./
48..57   0123456789
58..64   :;<=>?@
65..90   ABCDEFGHIJKLMNOPQRSTUVWXYZ
91..96   [\]^_`
97..122  abcdefghijklmnopqrstuvwxyz
123..126 {|}~
```

Each row lists consecutive characters/codes. Escapes in string literals: `\n` newline, `\t` tab, `\\` backslash, `\'` quote.

06 One-liners · list comprehensions

```
[f(x) for x in it]              # list
[f(x) for x in it if keep(x)]   # filter
[A(x) if test(x) else B(x) for x in it]
[x for row in matrix for x in row]
```

Trailing `if` keeps/drops items; `if...else` before `for` selects the output value. Nested `for`s run left to right, as nested loops.

```
[(x,y) for x in a for y in b]
dict([(k,f(v)) for k,v in d.items()])
```

07 Lists · list

Mutable; ordered; duplicates allowed.
`a = []` / `list()` • `[x]` is a valid singleton.
`list(it)` converts an iterable; `[0]*n` gives n zeros.

`a[i] = x` replaces; `a[i:j] = it` replaces a slice (length may change); `del a[i:j]` deletes a slice.

```
a.append(x)      # add x as ONE item
a.extend(it)     # add EACH item from it
a.insert(i,x)    # insert before index i
a.remove(x)      # delete FIRST equal x
a.pop()          # delete + RETURN last item
a.pop(i)         # delete + RETURN item i
a.clear()        # remove everything
a.reverse()      # reverse IN PLACE
a.sort()         # sort IN PLACE; p2
```

All methods above return None, except pop. Missing `remove` → `ValueError`; invalid/empty `pop` → `IndexError`. `del a[i]`: delete by index.

`a.count(x)` → occurrence count.
`a.index(x)` / `a.index(x,start,stop)` → first index, or `ValueError`; optional positional bounds default to `0`, `len(a)`; stop excluded.

```
a = [1]; a.append([2,3]) # [1,[2,3]]
a = [1]; a.extend([2,3]) # [1,2,3]
a += [4]                 # mutates a
b = a + [5]              # new list
```

Copy · alias

`b = a` → same object. `b = a.copy()` / `a[:]` / `list(a)` → **shallow** copy; nested objects still shared.

```
a = [[1]]; b = a.copy()
b[0].append(2)    # a is also [[1,2]]
b[0] = [9]        # a stays [[1,2]]
```

`[row]*R` repeats references to the same row. Removing while iterating a list can skip items; a filtered new list avoids that.

08 Tuples · tuple

Immutable slots • `t = ()` / `tuple()`.
`(x,)` → one-item tuple; `(x)` → x itself.
`tuple(it)` converts; `list(t)` makes mutable list.

```
t = (1,2)
t = t + (3,)       # new tuple (1,2,3)
t.count(x)         # count occurrences
t.index(x)         # or t.index(x,start,stop)
```

`index` returns first index or raises `ValueError`. No `append`/`remove`/`sort`. A contained list can still mutate. Hashable only if every element is hashable.

09 Sets · set

Unique, unordered, mutable. `s = set()` is empty; `{}` is a dict. `{x}` is a singleton.
`set(it)` removes duplicates; no indexing/slicing.

```
set('banana')      # {'a','b','n'} (any order)
s.add(x)           # insert one; None
s.update(it)       # insert all; None
s.remove(x)        # KeyError if missing
s.discard(x)       # no error if missing
s.pop()            # remove/return arbitrary x
s.clear()          # empty; None
s.copy()           # shallow copy
```

`remove/discard` → None; empty `pop` → `KeyError`.
`x in s` • `len(s)` • `for x in s:` • `list(s)` / `tuple(s)` have unspecified order; `sorted(s)` → list.

```
A | B              # union
A & B              # intersection
A - B              # in A, not in B
A ^ B              # in exactly one
A <= B             # subset; < proper subset
A >= B             # superset; > proper
A == B             # same elements
```

Operators need sets. Elements must be hashable: numbers, strings, suitable tuples; no list/dict/set elements. Equal keys like 1 and True collapse.

10 Iteration · range · unpacking

```
for x in it: ...
for i in range(len(a)): ...
for i,x in enumerate(a, start=0): ...
for x,y in zip(a, b): ...
for k,v in d.items(): ...
```

`range(stop)` / `range(start,stop)` / `range(start,stop,step)` → reusable immutable integer sequence. Default start `0`, step `1`; stop excluded; step nonzero. `range(5,0,-1)` → `5,4,3,2,1`.

`zip(a,b,...)` → iterator of tuples, ends at shortest input. `enumerate` → iterator of `(index,item)`.

```
x,y = (2,3)        # exact item count
x,y = y,x          # swap
for (r,c),v in pairs: ...
```

Unpacking too many/few items → `ValueError`. Iterating a dict yields keys; a set has no index order.

IT5001 / PRACTICAL PYTHON

SORTING · FUNCTIONS · FILES · REUSABLE CODE PATTERNS

11 Sorting · reverse · keyed lookup

```
sorted(it, key=None, reverse=False)
a.sort(key=None, reverse=False)
```

`sorted` → **new list** from any iterable. `list.sort` → **None**, mutates list. Options key / reverse are keyword-only. Stable: ties keep input order. `reverse=True`: descending. key takes one item, returns its sort key.

```
sorted(words, key=len)   # by length
sorted(words, key=str.lower)
sorted(rows, key=lambda r:r[1])
# numeric score descending, name ascending
sorted(rows, key=lambda r:(-r[1],r[0]))
```

Dictionary → sorted keys / pairs / dict

```
sorted(d)                # keys by key
sorted(d, key=d.get)     # keys by value
sorted(d.items())        # pairs by key
sorted(d.items(), key=lambda kv:kv[1])
# value descending; ties keep input order
pairs = sorted(d.items(),
    key=lambda kv:kv[1], reverse=True)
ordered = dict(pairs)    # preserves order
```

No dict `.sort()`. Value ties keep insertion order unless a tie-breaker is supplied. Sort keys must be comparable.

```
a.reverse()              # list; None
a[::-1]                  # same sequence type
reversed(a)              # consumable iterator
```

Prefix match after sorting · adjacent pairs

```
b = sorted(words)
has_prefix = False
for i in range(len(b)-1):
    if b[i+1][:len(b[i])] == b[i]:
        has_prefix = True
        break
```

12 Functions as values · map / filter

Function values · one expression

```
square = lambda x: x*x
choose = lambda x: x if x > 0 else 0
(lambda a,b: a+b)(2,3)      # 5
def make_power(n):
    return lambda x: x**n
make_power(3)(2)            # 8
```

`f` is the function; `f(x)` is its result. Lambda body is one expression; no `return` inside it.

Map transforms · filter keeps original items

```
map(f, it, *more_iterables)
list(map(int, ["2","3"]))   # [2,3]
list(map(lambda x,y:x+y, [1,2], [10]))
# [11]; stops at shortest iterable
filter(predicate, it)
list(filter(lambda x:x>0, [-2,0,3]))
# [3]; only truthy predicate results survive
list(filter(None, [0,1,"",2])) # [1,2]
```

`map` / `filter` → **consumable iterators**, no indexing. `for x in it`, `list(it)` or `tuple(it)` consumes remaining items; second conversion → empty.

```
m = map(abs, [-2,3])
list(m)                  # [2,3]
list(m)                  # []
```

`all(it)` → all truthy (`all([])` → True). `any(it)` → some truthy (`any([])` → False). `sum([x for x in a if x>0])` → positive total.

Reduce / accumulate · course loop form

```
acc = initial
for x in it:
    acc = combine(acc,x)
```

`initial`: 0 for sum, 1 for product, `""` for text. `reduce()` itself is not taught in these slides and requires an import; this loop uses course constructs.

13 Text files · input / output

```
with open(path, 'r') as f:
    for line in f:
        word = line.rstrip('\n')
        parts = word.split(',')
```

`open(path, mode="r")` → file; with closes it. `r`: read existing • `w`: create/truncate • `a`: create/append • `+`: read and write • `b`: binary; default text.

`f.read(n)` / `f.readline(n)` → str (text / next line), at most n chars; omit n or use `-1` for no limit. Newlines retained; EOF → `""`. `f.readlines()` → remaining lines as list. Reading advances position.

```
with open(path, 'w') as f:
    f.write(str(value) + '\n')
```

`f.write(s)` → character count; no automatic newline. `input()` / `input(prompt)` → str without final newline.
`print(*items, sep=" ", end="\n")` → None.

Simple `split(",")` does not parse quoted CSV fields. In slides: `import csv`; `csv.reader(f)` → row iterator, each row a list of strings. Requires import.

14 Functions · branching · recursion

```
def f(a, b=0):
    if a > b:
        return a
    elif a == b:
        return 0
    else:
        return b
f(2, b=3)               # 3
```

`return` exits the function; no return / bare `return` → None. `return x,y` returns a tuple.
`break`: innermost loop exits; `continue`: next iteration; `pass`: does nothing.

Parameters/local assignments stay local. Mutating a passed list/dict changes that object; rebinding the parameter does not rebind the caller's name.

Assigning a name anywhere in a function makes it local unless declared `global name`. Reading that local before assignment → `UnboundLocalError`. Mutable default arguments are reused between calls.

Sequence recursion · base before indexing

```
def total(a):
    if not a:
        return 0
    return a[0] + total(a[1:])
```

Each call needs a smaller problem and a returned result. Long recursion can raise `RecursionError`; slicing copies sequences.

Nested lists · preserve structure / flatten

```
def deepmap(f, x):
    if not isinstance(x, list):
        return f(x)
    return [deepmap(f,v) for v in x]

def flatten(x):
    if not isinstance(x, list):
        return [x]
    out = []
    for v in x:
        out.extend(flatten(v))
    return out
```

`deepmap(lambda x:x, a)` copies nested **lists**; other mutable leaves remain shared.
`len(flatten(a))` counts leaves. `isinstance(x, (list,tuple))` also recognizes tuples.

15 Search · selection sort · merge sort

Binary search · ascending sorted sequence

```
def binary(a, x):
    lo, hi = 0, len(a)-1
    while lo <= hi:
        mid = (lo+hi)//2
        if a[mid] == x:
            return True
        if a[mid] < x:
            lo = mid+1
        else:
            hi = mid-1
    return False
```

Selection sort · returns a new list

```
def selection(a):
    b, out = list(a), []
    while b:
        smallest = b[0]
        for x in b:
            if x < smallest:
                smallest = x
        b.remove(smallest)
        out.append(smallest)
    return out
```

Merge sort · returns a new list

```
def merge(a, b):
    out, i, j = [], 0, 0
    while i < len(a) and j < len(b):
        if a[i] <= b[j]:
            out.append(a[i])
            i += 1
        else:
            out.append(b[j])
            j += 1
    return out + a[i:] + b[j:]
def merge_sort(a):
    if len(a) < 2:
        return list(a)
    k = len(a)//2
    return merge(merge_sort(a[:k]),
                 merge_sort(a[k:]))
```

Merge uses indices so items need not shift on `pop(0)`; `<=` takes the left item on ties. Binary search uses bounds so it does not copy sublists.

Word fragments · split points + set lookup

```
pieces = set(fragments)
matches = []
for i in range(1, len(word)):
    left, right = word[:i], word[i:]
    if left in pieces and right in pieces:
        matches.append((left, right))
```

16 2D lists · matrices · grids

`m[r][c]`: row first, column second. Rectangular grid: `R = len(m); C = len(m[0]) if m else 0`.

```
m = [[0]*C for _ in range(R)]
b = [row[:] for row in m] # copy each row
# NOT [[0]*C]*R: rows would be shared
for r in range(R):
    for c in range(C):
        value = m[r][c]
```

Comprehension recipes

```
col = [row[c] for row in m]
flat = [x for row in m for x in row]
transposed = [[m[r][c] for r in range(R)]
              for c in range(C)]
# A + B, same rectangular dimensions
added = [[A[r][c]+B[r][c]
          for c in range(C)]
         for r in range(R)]
# n x n identity matrix
I = [[1 if r==c else 0 for c in range(n)]
     for r in range(n)]
```

Position conversions (`C > 0`): `r = p//C`, `c = p%C`, `p = r*C+c` (zero-based). One-based position: use `p-1`, then add 1 when converting back.

`(r//s + c//s) % 2` → alternating block parity (`s > 0`); `"#"` for 0, `" "` for 1.

Bounds before access: `0 <= r < R and 0 <= c < C`. Negative indices wrap to the opposite edge. For ragged rows use `len(m[r])` per row.

17 BFS / flood fill · island perimeter

Rectangular 0/1 grid; start `(r,c)` is unvisited land. Shared seen is a set of coordinates. Queue is a list plus a head index; no import or input mutation.

```
def island(m, r, c, seen):
    R, C = len(m), len(m[0])
    q, head, p = [(r,c)], 0, 0
    seen.add((r,c))
    while head < len(q):
        r,c = q[head]
        head += 1
        for dr,dc in ((1,0),(-1,0),
                      (0,1),(0,-1)):
            x,y = r+dr,c+dc
            if not (0 <= x < R
                    and 0 <= y < C):
                p += 1
            elif m[x][y] == 0:
                p += 1
            elif (x,y) not in seen:
                seen.add((x,y))
                q.append((x,y))
    return p
```

Each water/boundary neighbour adds one edge. Mark land seen **when enqueuing**, so each cell enters once. Only horizontal/vertical connections count.

```
def all_perimeters(m):
    seen, out = set(), []
    for r in range(len(m)):
        for c in range(len(m[r])):
            if m[r][c] == 1:
                if (r,c) not in seen:
                    out.append(
                        island(m,r,c,seen))
    return out
p = all_perimeters(m)
# total: sum(p); maximum: max(p, default=0)
```

18 Objects · classes · inheritance

```
class Account:
    def __init__(self, name, balance):
        self.name = name
        self.balance = balance
    def deposit(self, amount):
        self.balance += amount
        return self.balance
a = Account("Ann", 10)
a.deposit(5)            # 15; a.balance = 15
class Child(Account):
    def __init__(self, name, balance):
        super().__init__(name, balance)
    def deposit(self, amount):
        return super().deposit(amount)
```

`self.attr` persists on the object; plain `attr` is local. `obj.method(x)` passes `obj` as `self`. `__init__` must return None. Overriding a method replaces inherited behaviour.

`isinstance(obj,C)` includes subclasses; `type(obj) is C` is exact. `self.__x` becomes `_ClassName__x`, not fully private.

```
class Both(A, B):
    def __init__(self, x):
        A.__init__(self, x)
        B.__init__(self)
```

Above: slide pattern for independent parents. `super()` follows method resolution order; it does not automatically call every parent initializer.